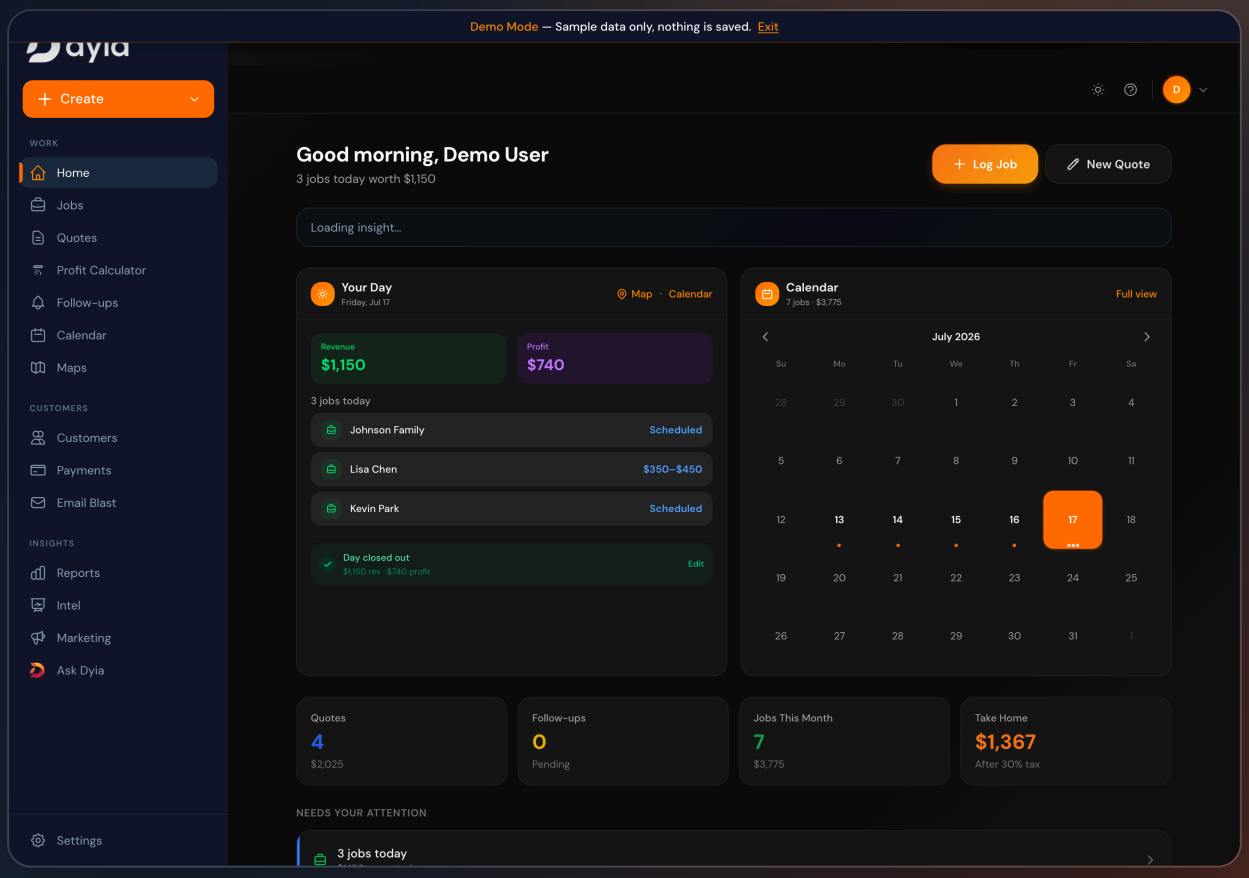
TECHNICAL SHOWCASE · FORENSIC PORTFOLIO

dyia — the operating system for service businesses

A multi-tenant SaaS that turns day-to-day service work — jobs, quotes, payments, follow-ups — into tracked profit, with a human-in-the-loop AI assistant on top.



Prepared by **Dev Ganugapenta** — Founder, InitDev · July 2026
All screenshots captured live in Demo Mode (sample data only — no real customer information).

01 Executive assessment

dyia is a production-oriented operations platform for owner-operated service businesses (junk removal, lawn care, house cleaning). It replaces spreadsheets and disconnected tools with one system for jobs, profit, quotes, customer payments, follow-ups, reporting, and an AI assistant.

70 API ROUTE HANDLERS	**29** POSTGRES TABLES	**21** TYPED AI TOOLS	**11** EXTERNAL INTEGRATIONS
50 SQL MIGRATIONS	**55** REACT COMPONENTS	**4** SCHEDULED JOBS	**229** GIT COMMITS

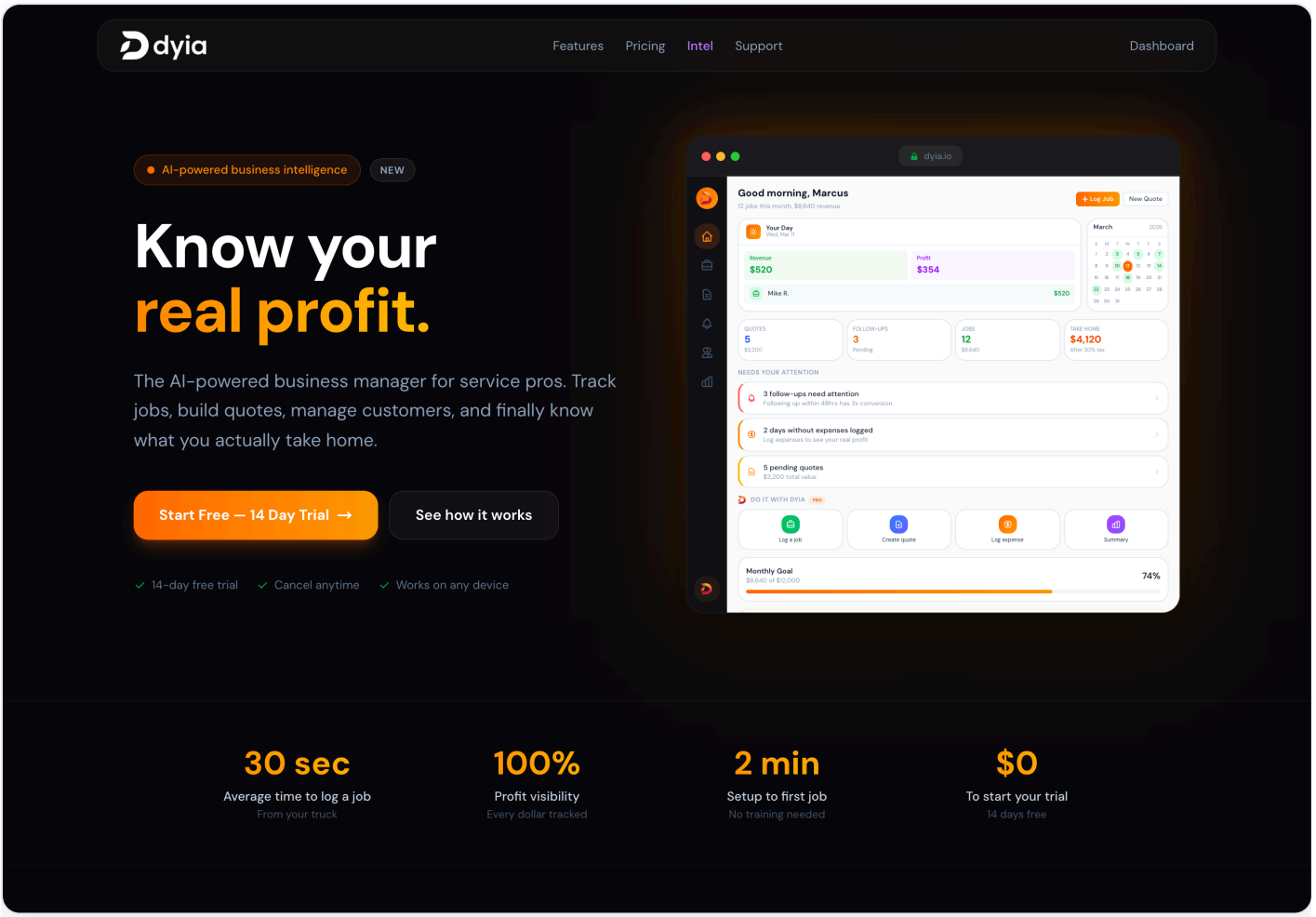
Maturity: connected to a live Stripe account with real subscriptions and merchant payouts, Clerk auth, and managed Supabase Postgres. Exact revenue/user figures are treated as unverified and omitted.

Why it matters: it moves real money (subscriptions + platform-fee customer payments), encodes how a service business actually runs (per-job profit after expenses, tax set-aside), and layers safe AI automation — well beyond a CRUD app or thin API wrapper.

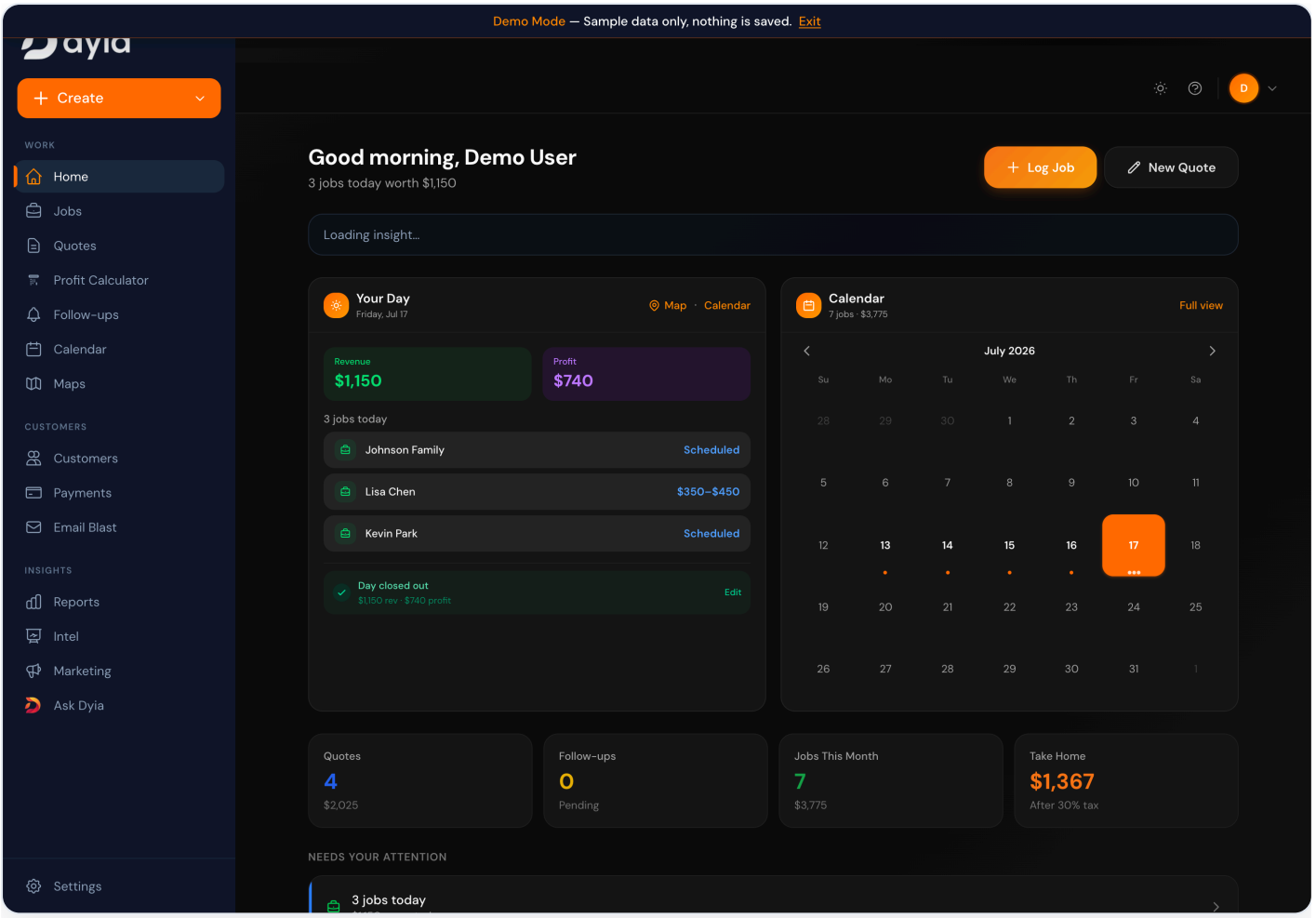
Evidence labels used throughout: IMPLEMENTED present & verifiable PARTIAL present but incomplete INFERRED strongly suggested by config PROPOSED recommended, not built

02 Live product walkthrough

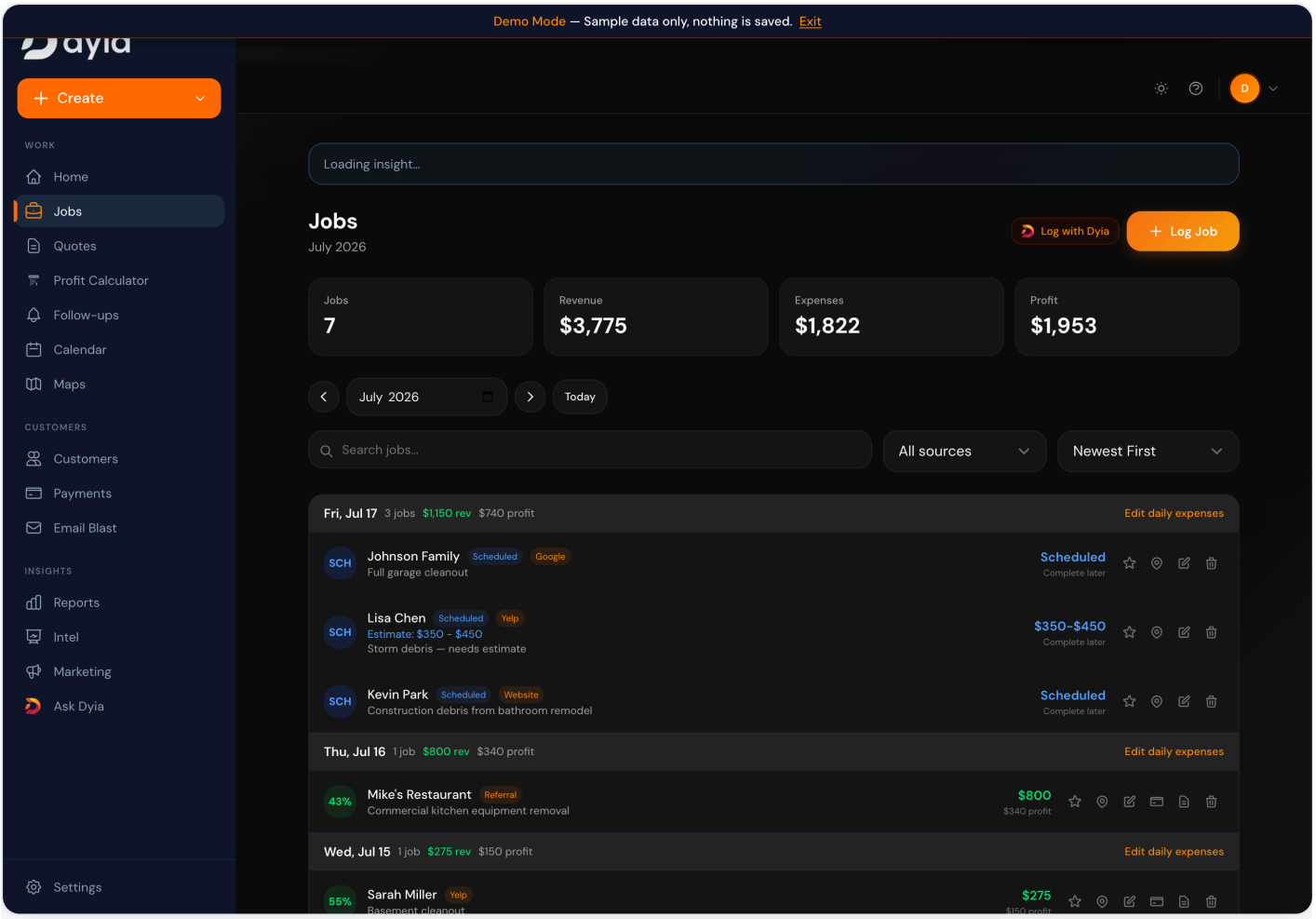
Purposeful captures from the running app in Demo Mode. Each caption states what it proves.



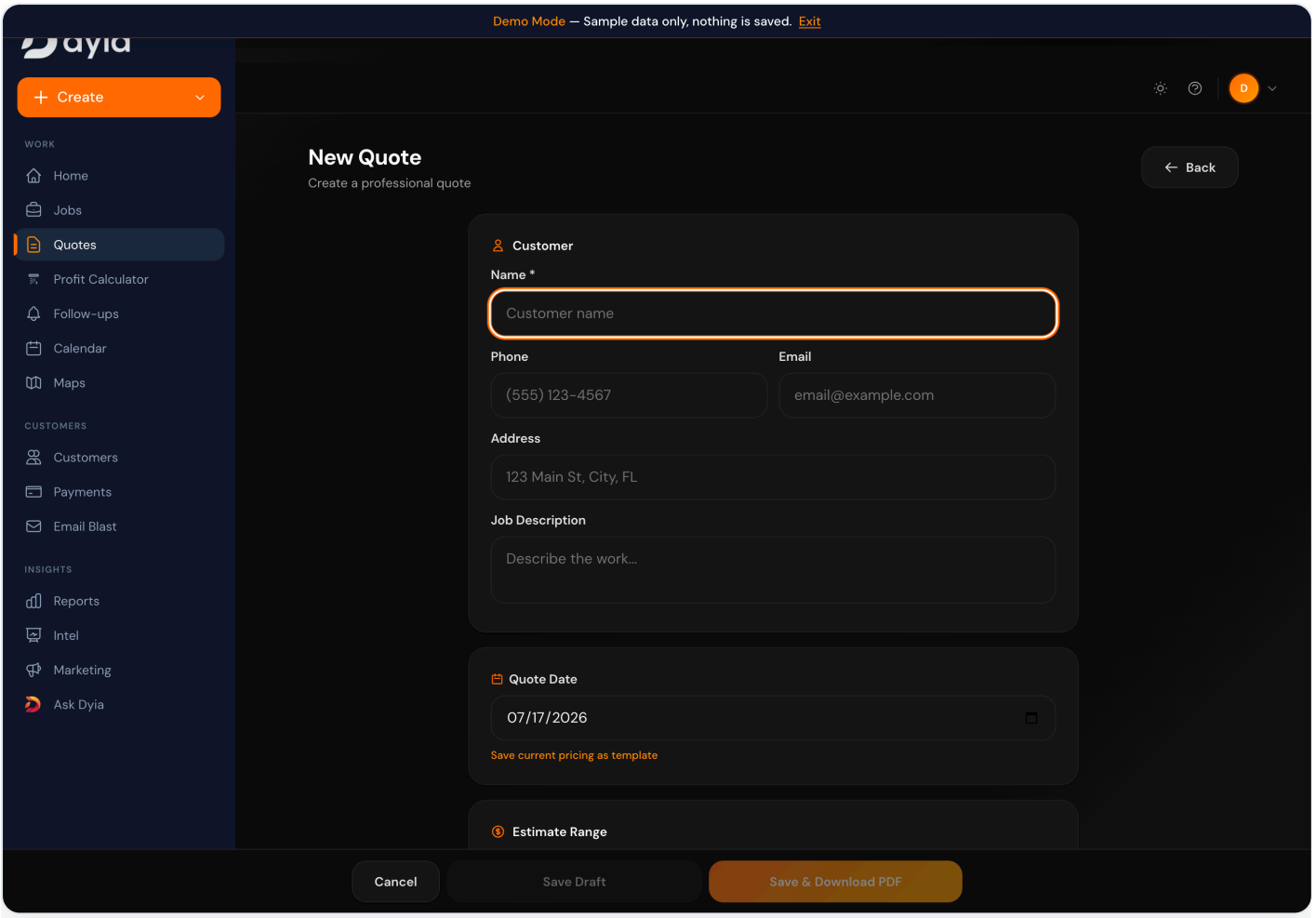
IMPLEMENTED Public marketing/entry with a self-serve demo path — full public/authenticated split and real onboarding funnel.



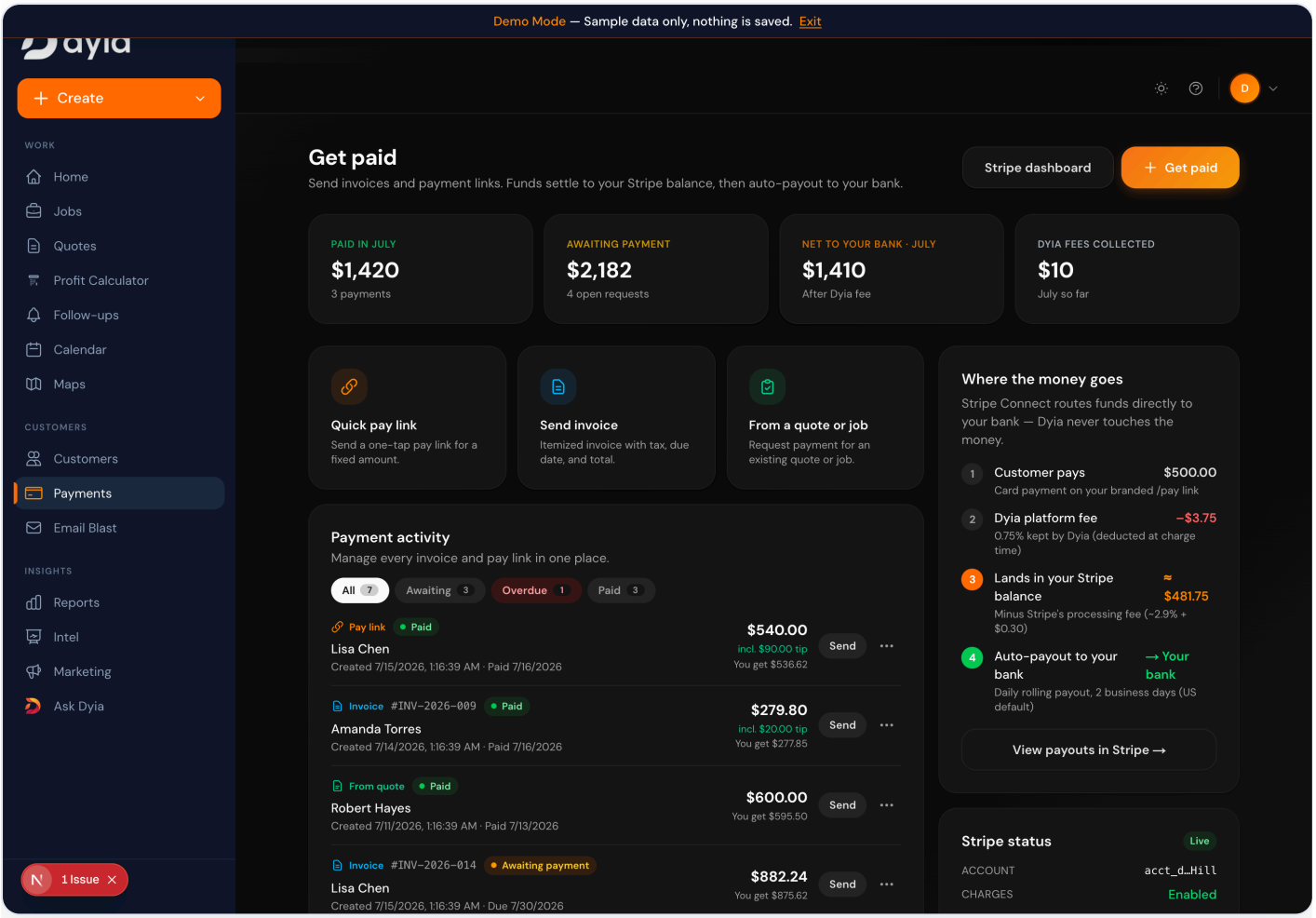
IMPLEMENTED Authenticated home: per-tenant revenue/profit, monthly-goal progress, and schedule — server-side aggregation of computed business state.



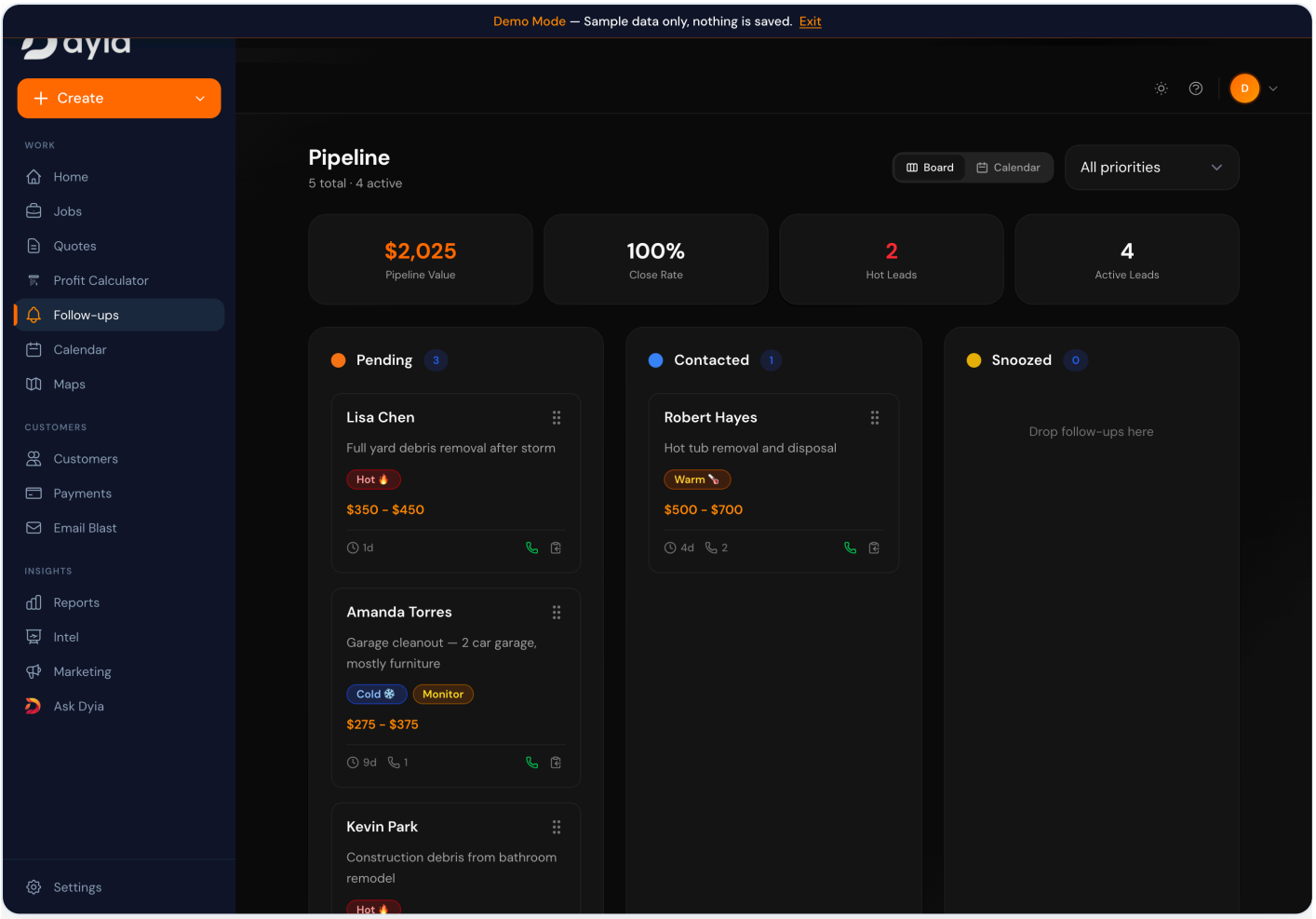
IMPLEMENTED Job ledger with per-job revenue, itemized expenses, and computed profit/margin — transactional CRUD plus financial computation.



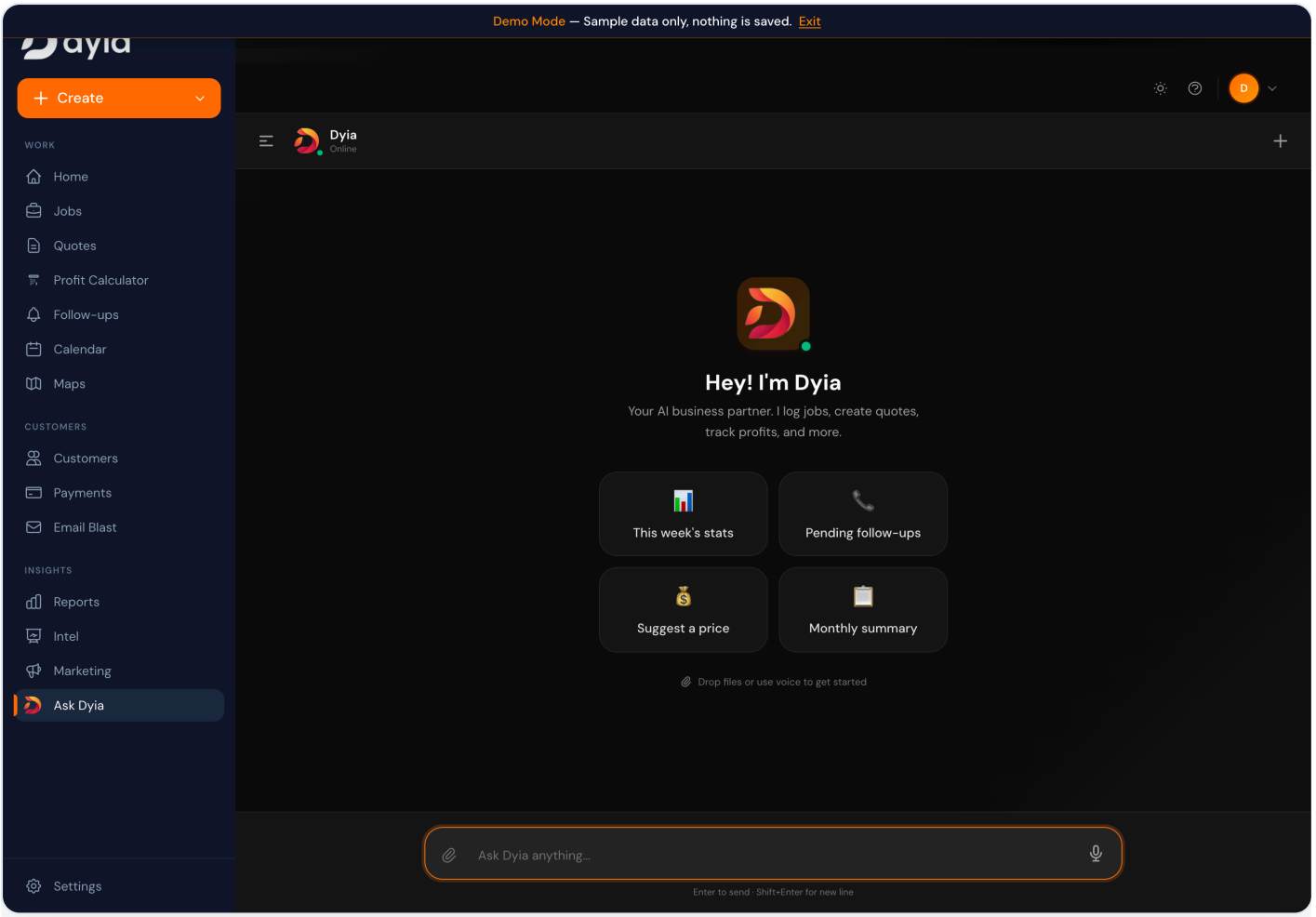
IMPLEMENTED Structured quote builder with line-item pricing and PDF export — server-side document generation.



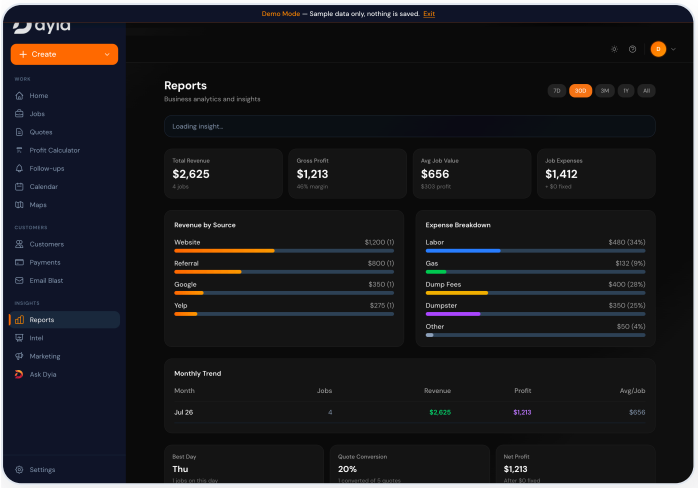
IMPLEMENTED Payments hub: pay links, itemized invoices, and quote/job-linked requests with a transparent platform-fee and payout model — real Stripe Connect money movement.



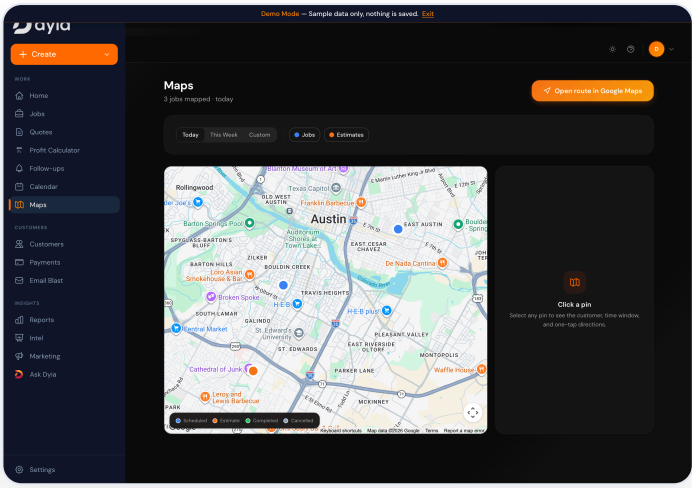
IMPLEMENTED Follow-up pipeline (Pending / Contacted / Snoozed) with priority — an explicit status state machine driving an operational workflow.



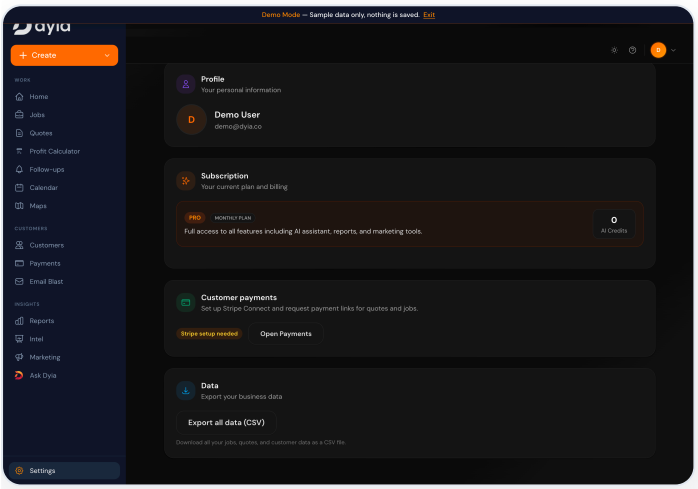
IMPLEMENTED AI assistant workspace wired to the same backend tools that power the UI — conversational interface over the operational data.



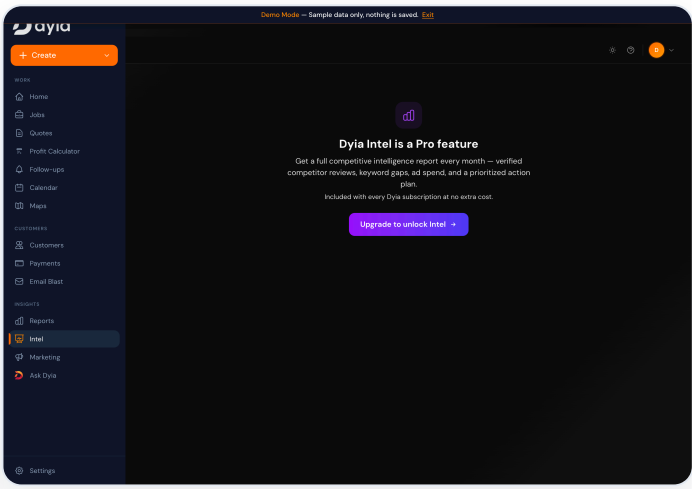
IMPLEMENTED Reporting: revenue by source, expense breakdown, margins, trend — analytics from the operational model.



IMPLEMENTED Geocoded jobs on a live map with route planning — third-party geospatial integration tied to records.

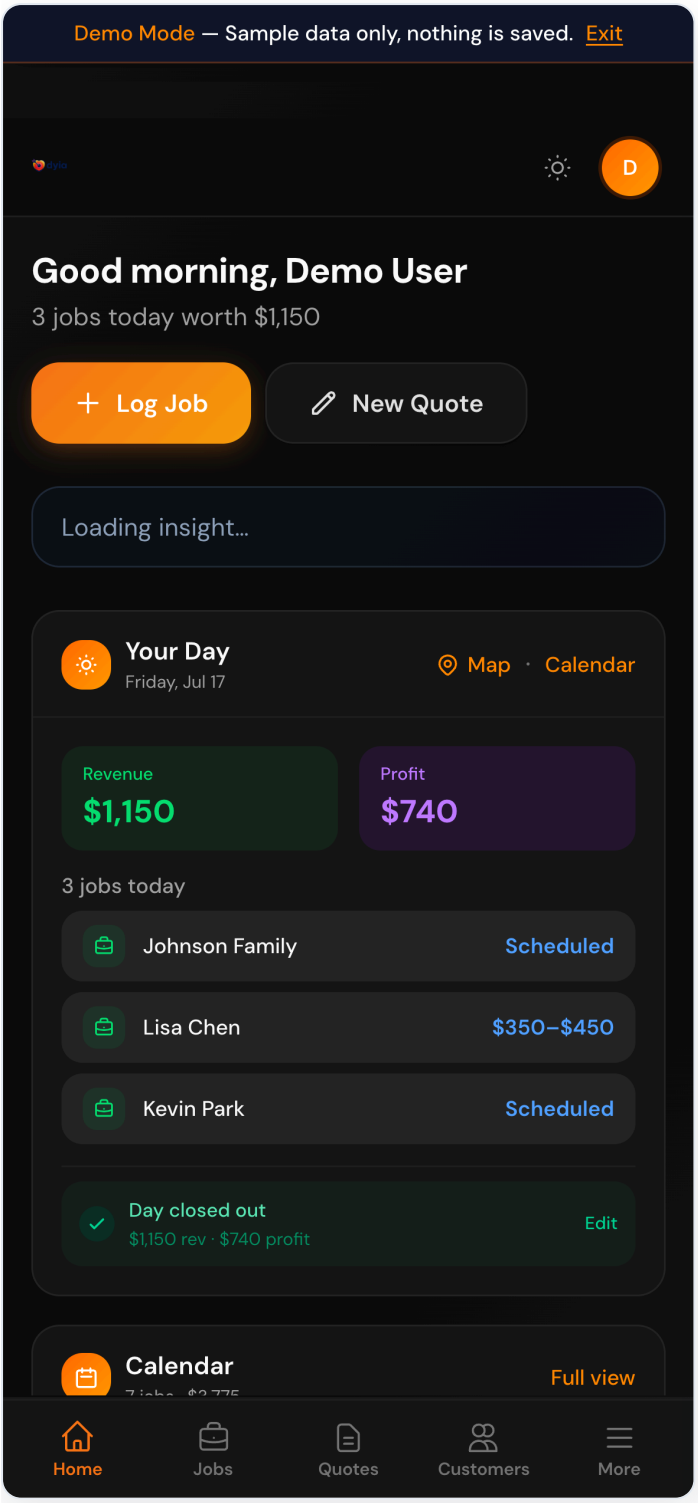


IMPLEMENTED Account/subscription surface — tier, business config, and billing controls.

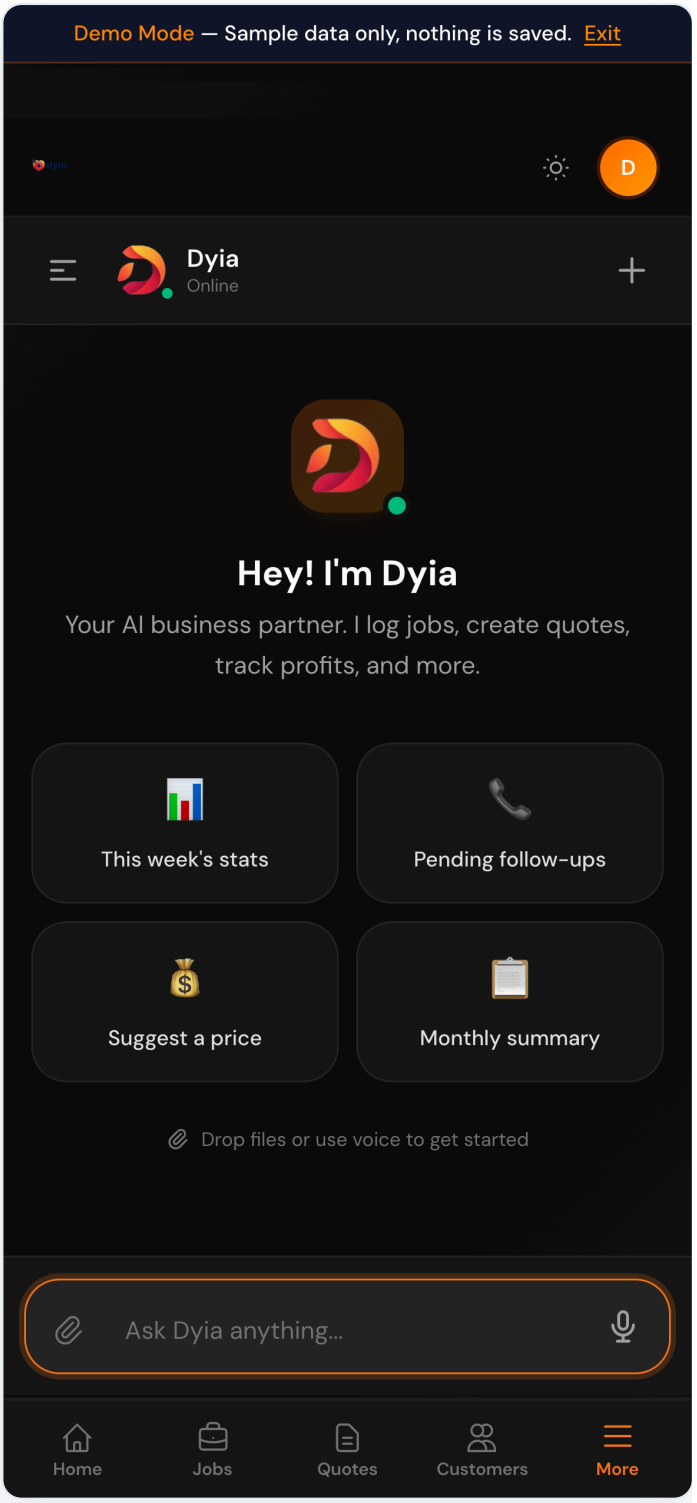


PARTIAL Dyia Intel — a Pro-gated competitive-intelligence feature; tier feature-gating enforced in the UI.

Mobile (field-first)



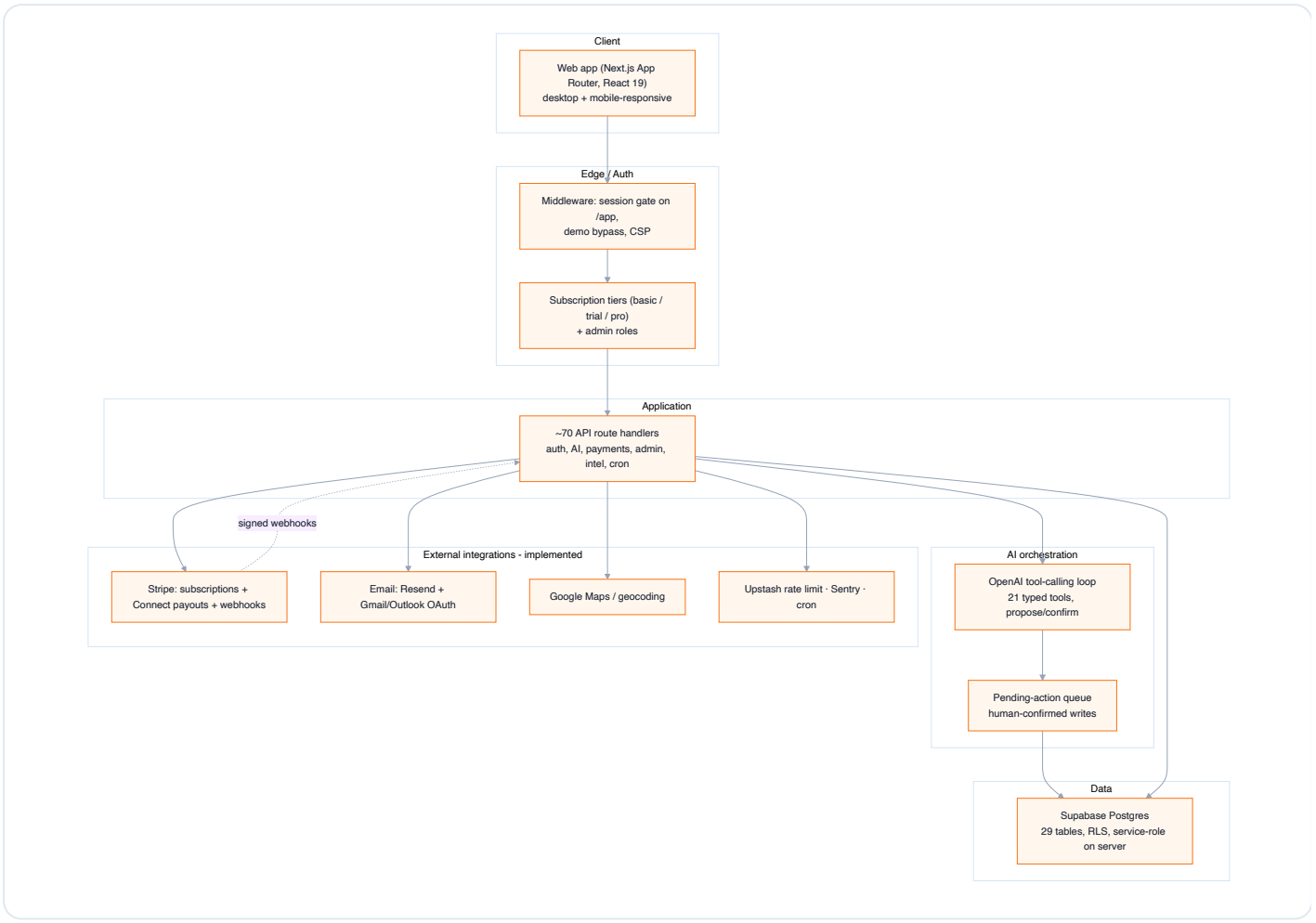
IMPLEMENTED Mobile dashboard — the same product reflows for operators in the field.



IMPLEMENTED Full AI assistant on a phone — not a cut-down view.

03 System architecture

One Next.js App Router application serves the public site, the authenticated product, and the API. Server route handlers own privileged work; the browser reads under row-level security.

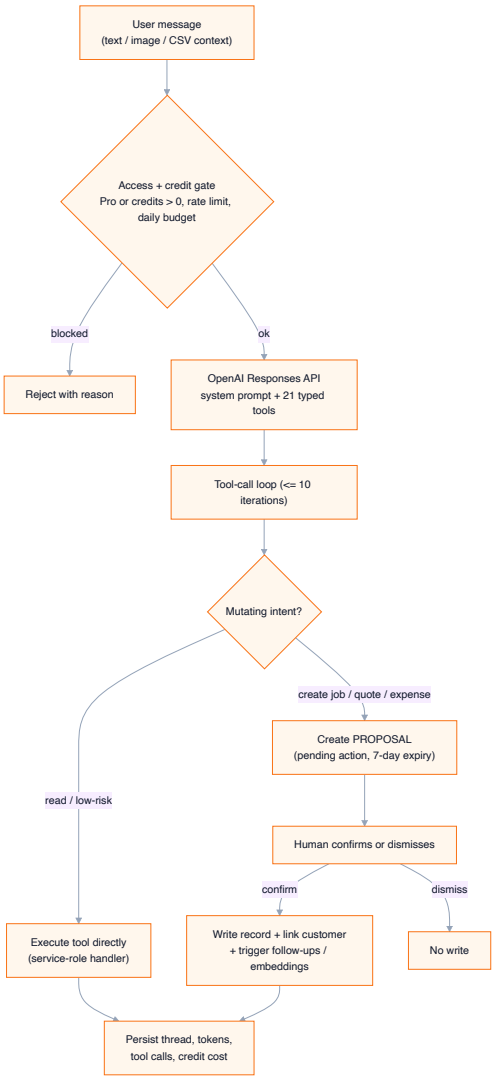


Request and data flow across layers. Solid = verified in code; dashed = signed webhook callback. Integrations shown are all present in the codebase.

> **Trust boundary:** the browser uses an anon Supabase client under RLS; privileged work (AI writes, webhooks, admin) runs server-side with a service-role client. Trust stays on the server.

04 AI system design

The AI layer is an orchestrator, not a chat wrapper. A natural-language message becomes typed tool calls in a bounded loop; any state-changing intent is routed through explicit human confirmation.

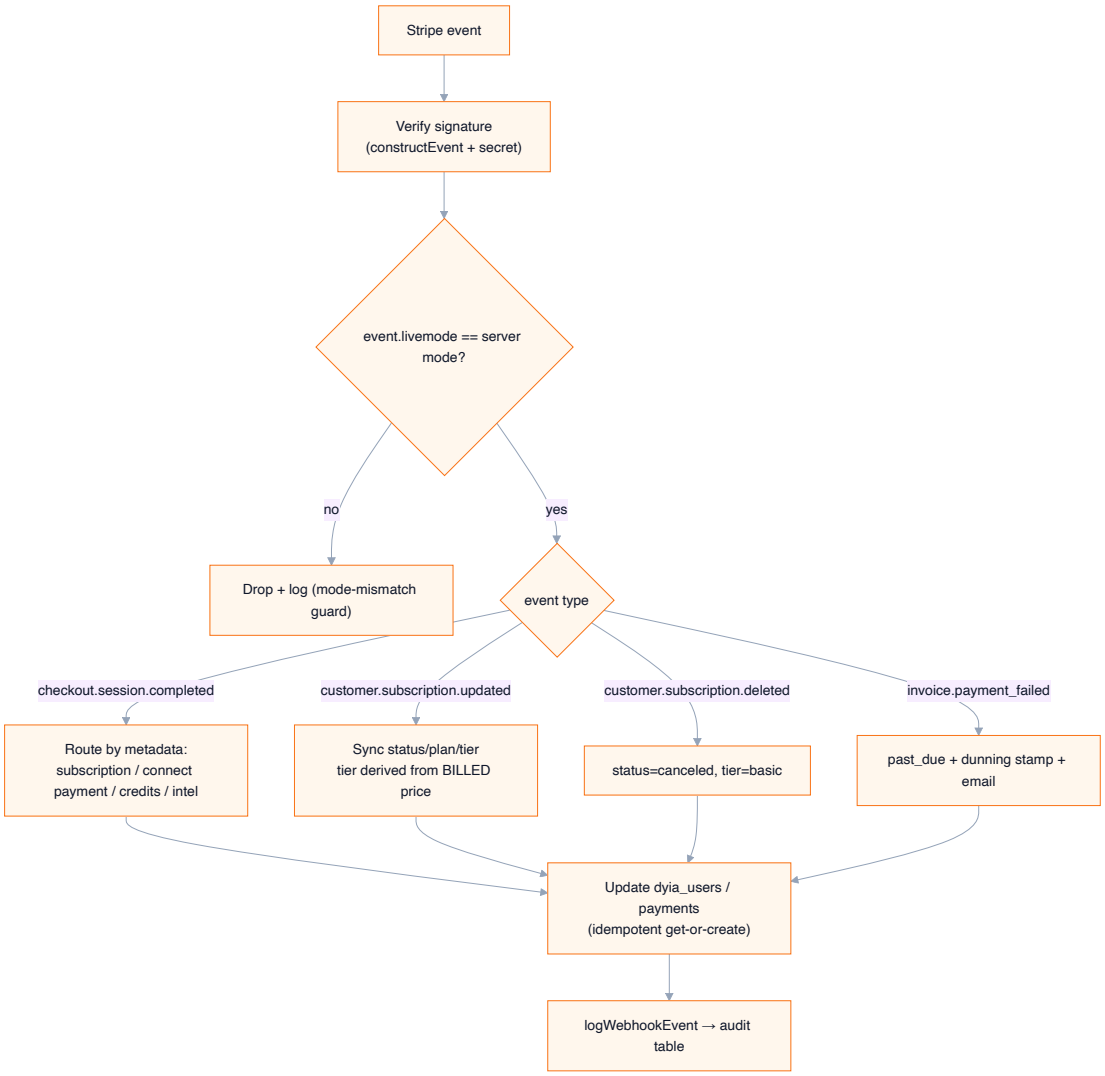


One message becomes a guarded, persisted, optionally human-approved action. Mutating intents become proposals; reads execute directly.

- **21 strictly-typed tools** with a bounded loop (≤10 iterations). IMPLEMENTED
- **Propose/confirm**: create-job/quote/expense become pending actions requiring user confirmation before any write. IMPLEMENTED
- **Guardrails**: access/credit gate, per-IP rate limit, daily budget, strict JSON schemas. IMPLEMENTED
- **pgvector** job-similarity embeddings feed AI context. IMPLEMENTED

05 Payments & money-state integrity

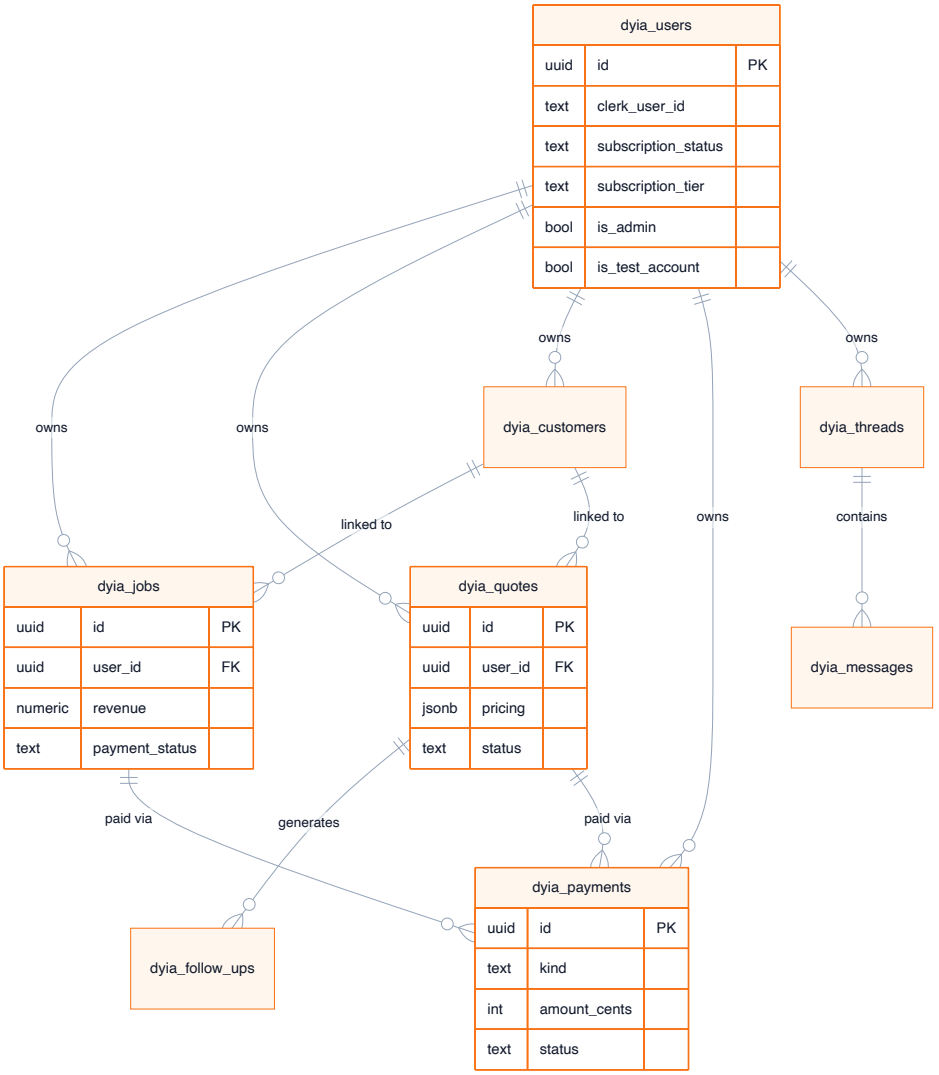
Two payment systems: platform subscriptions and Stripe Connect customer payments. Both are reconciled through signed, idempotent webhooks.



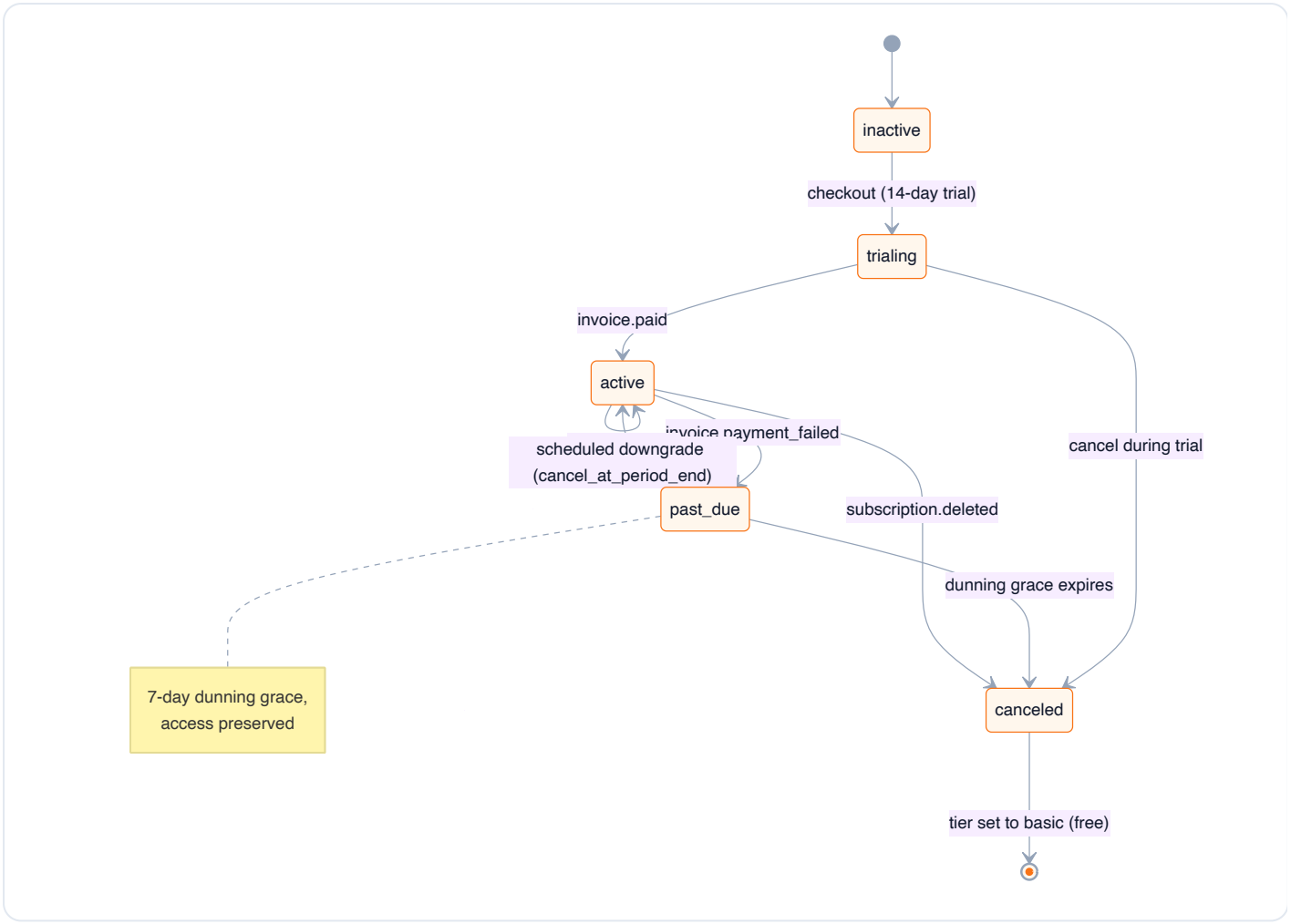
Webhook handling: signature verification, a livemode-mismatch guard, event routing, and idempotent DB updates with an audit log.

Correctness details: subscription tier is derived from the *billed price*, not client checkout metadata (a real bug that was fixed and guarded); payment rows use idempotent get-or-create; a livemode guard prevents test-mode events from rewriting live rows.

06 Data model & subscription state



Focused ERD of the core domain entities (of 29 total tables). Every record is tenant-scoped to a user.



Subscription lifecycle with a 7-day dunning grace and scheduled-downgrade (cancel_at_period_end) semantics mirrored from Stripe.

07 Most advanced implementations

#	Capability	Why it is hard	Evidence	
1	Human-in-the-loop AI tool orchestration	Bounded agentic loop + write-gating keeps a human in control of the system of record	`openai/functions.ts · 008_pending_actions.sql`	IMPL
2	Idempotent Stripe webhook + money-state machine	Replay safety, livemode isolation, price-authoritative tiering	`stripe/webhook/route.ts · stripe-mode.ts`	IMPL
3	Stripe Connect merchant payments + platform fees	Money movement to third-party banks, reconciled via webhooks	`api/payments/** · Payments.tsx`	IMPL
4	Multi-tenant isolation + subscription gating	RLS + a single access computer used everywhere, unit-tested	`018_rls_policies.sql · subscription.ts`	IMPL
5	Dunning grace + scheduled-downgrade semantics	Accurate billing state without per-render Stripe calls	`037_payment_failed_grace.sql · 040_cancel_at_period_end.sql`	IMPL
6	Vector job similarity (pgvector)	Embedding pipeline + backfill for AI context	`007_job_embeddings.sql · backfill-embeddings.ts`	IMPL
7	Trade-agnostic quoting + domain pricing	Per-trade templates, per-job profit after expenses + tax	`quotes/templates.ts · QuoteBuilder.tsx`	IMPL
8	Operational admin control plane	Users, metrics w/ test exclusion, webhook audit, plan switching	`api/admin/** · AdminPanel.tsx`	IMPL
9	Dyia Intel competitive research	Cron-driven, Pro-gated external research	`lib/intel/** · cron/intel-monthly`	PARTIAL

08 Engineering judgment demonstrated

Evidence-based capabilities the repository proves. Confidence reflects how directly the code demonstrates each — not any personal judgment.

Capability	Evidence	Why demanding	Confidence
Probabilistic/AI-system reasoning	Propose/confirm + typed tools + bounded loop	Safe agentic writes are subtle	High
Financial correctness & idempotency	Webhook idempotency, livemode guard, price-authoritative tier	Money bugs are high-cost, adversarial	High
Data modeling & tenancy	29 tables, RLS, status enums, audit columns	Isolation + integrity easy to get wrong	High
Systems thinking across surfaces	70 routes: auth, AI, payments, admin, cron	Holding a whole product in mind	High
Debugging / production hardening	QA-round guards, self-healing Stripe customer resolution	Reading real failures & hardening	High
Reliability / operational thinking	Dunning grace, rate limiting, webhook audit, cron	Anticipating partial failure & abuse	Moderate
Product & business reasoning	Tiers, platform fees, retention loop	Translating business to schema/flows	Moderate
Ownership under ambiguity	Breadth by a small contributor set (229 commits)	Coordinating everything	Moderate

09 Security, testing & production readiness

Security & authorization IMPLEMENTED

- Session gate on `/app`; admin APIs guarded by `requireAdmin`.
- RLS on client reads; service-role clients confined to server routes (`018_rls_policies.sql`).
- Webhook signature verification (Stripe `constructEvent`; Clerk via svix).
- Stripe livemode-mismatch guard; per-IP rate limiting (`rate-limit.ts`).

Testing & verification

Check	Command	Result
Lint	`npm run lint`	Pass — 0 errors, 48 warnings
Unit/integration	`npm test`	Pass — 53 tests (3 files)
Production build + types	`npm run build`	Pass

> **Honest limitation:** automated test coverage is thin (3 files, concentrated on subscription and maps logic). Manual QA is evidenced in code (multiple "QA round" hardening passes) but there is no committed end-to-end suite. This is a real gap, stated plainly rather than inflated.

Operations INFERRED/IMPLEMENTED

- 4 scheduled cron jobs (trial reminders, follow-ups, weekly insights, monthly intel).
- Webhook-event audit table; Sentry error tracking (inferred from config).
- Admin control plane for support: users, metrics, plan switching, cancellations.

10 Résumé-ready achievements

Technical / product engineering

- Built a multi-tenant Next.js 16 / React 19 SaaS — 70 API routes, 29 Postgres tables, 11 integrations — across auth, payments, AI, and operations.
- Designed a human-in-the-loop AI assistant: 21 typed OpenAI tools in a bounded loop where state-changing actions are proposed and require confirmation before any DB write.
- Implemented idempotent, signature-verified Stripe webhooks with a livemode-mismatch guard; hardened tiering to derive from the billed price, not client metadata.
- Shipped Stripe Connect merchant payments (pay links, invoices, quote/job-linked) with platform fees and webhook reconciliation.
- Modeled multi-tenant data with RLS, status enums, audit columns, and pgvector job-similarity embeddings.

Founder / technical leadership

- Primary engineer across frontend, backend, data, payments, AI, and deployment for a production-oriented SaaS (229 commits).
- Owned billing integrity end to end: diagnosed a tier/price mismatch, built read-only reconciliation + overcharge-audit tooling, and drove customer remediation.
- Built an operational admin control plane and a retention loop (cancellation feedback → retargeting queue) tied to real churn data.

Interview narratives

1. **Hardest architecture:** safe agentic AI writes via a bounded tool loop + pending-action queue.
2. **Reliability:** Stripe cross-mode contamination — livemode guard + self-healing customer resolution.
3. **Best product decision:** platform-fee merchant payments (product becomes part of the money flow).
4. **End-to-end ownership:** the billing tier incident — detection, tooling, root cause, code fix, refunds.
5. **Lesson:** never trust client metadata for money state — the billed price is authoritative (now enforced + guarded).

11 Limitations, risks & evidence index

Honest limitations & risks

- Thin automated test coverage; no committed E2E suite. PARTIAL
- Dyia Intel depends on external research + model calls; treat as PARTIAL .
- Anthropic + Sentry present but usage not fully verified. INFERRED
- Business metrics (users, MRR, churn) are UNVERIFIED and omitted from claims.

Target roles this evidences

Senior/founding full-stack engineer · product engineer · AI application engineer · technical lead · solutions architect · technical co-founder.

Evidence index (selected)

Claim	Path
AI tools + handlers	`src/lib/openai/functions.ts, handlers.ts`
Pending actions (human-in-loop)	`supabase/migrations/008_pending_actions.sql · api/pending-actions`
Stripe webhook + mode guard	`api/stripe/webhook/route.ts · lib/stripe-mode.ts`
Connect payments	`api/payments/** · components/app/Payments.tsx`
Subscription logic + tests	`lib/subscription.ts · lib/subscription.test.ts`
RLS	`supabase/migrations/018_rls_policies.sql`
Embeddings	`007_job_embeddings.sql · scripts/backfill-embeddings.ts`
Admin control plane	`lib/admin.ts · api/admin/** · AdminPanel.tsx`

Full detail in `TECHNICAL_EVIDENCE.md`. Counts reproduced from `report-data.json`. Verification log in `BUILD_NOTES.md`.